

1 INTRODUCTION

Clock gating is a power reduction technique that has been used successfully in the custom ASIC domain. Clock and logic signal power are saved by temporarily disabling the clock signal on registers whose outputs do not affect circuit outputs. We consider and evaluate FPGA clock network architectures with built-in clock gating capability and describe a flexible placement algorithm that can operate with various gating granularities (various sizes of device regions containing clock loads that can be gated together). Results show that depending on the clock gating architecture and the fraction of time clock signals are enabled, clock power can be reduced by over 50%, and results suggest that a fine granularity gating architecture yields significant power benefits.

The Achilles' Heel of field-programmable gate arrays (FPGAs) in mobile electronics applications is their high power consumption. The circuit overhead associated with logic flexibility and re-programmability leads to longer wire lengths and more dynamic power vs. custom chips. A recent work suggests that dynamic power in FPGAs is 7-14 X higher than in custom ASICs for implementing a given circuit. Large strides are needed on the power front to enable deployment of programmable logic in the handheld battery powered devices pervasive in society today.

We focus on reducing dynamic power consumption, which is power consumed due to transitions on circuit signals, defined by

$$P_{avg} =$$

$$\frac{1}{2} \sum_{i \in \text{nets}} f_i \cdot C_i \cdot V^2, \text{ where } f_i \text{ is the}$$

average toggle rate on net i (net i 's switching activity), C_i is the capacitance of net i and V is the supply voltage. Specifically, we focus on reducing the power consumed by the FPGA's clock network, which prior work has shown to comprise up to 22% of power in an FPGA tile. Reducing the other components of dynamic power, e.g. within the logic cells and general interconnect, has been considered recently by other works.

Clock gating is a way of reducing switching activity on circuit signals, and it entails temporarily disabling the clocking of specific registers when the outputs of those registers are inconsequential to circuit outputs. Long used successfully in the ASIC domain, it was applied recently for FPGAs, albeit in a limited way. We consider architectures for clock gating in FPGAs and propose incorporating gating capabilities at points within the clock distribution network. The clock network has a tree structure, comprising wire segments and switches that distribute clock signals with low skew. Closing a switch amounts to driving a clock signal into a particular region of the device.

We propose making such switches capable of receiving an enable signal that when deasserted, disables clocking within the corresponding region. To our knowledge, this is the first work to consider a range of architectures for clock gating in FPGAs.

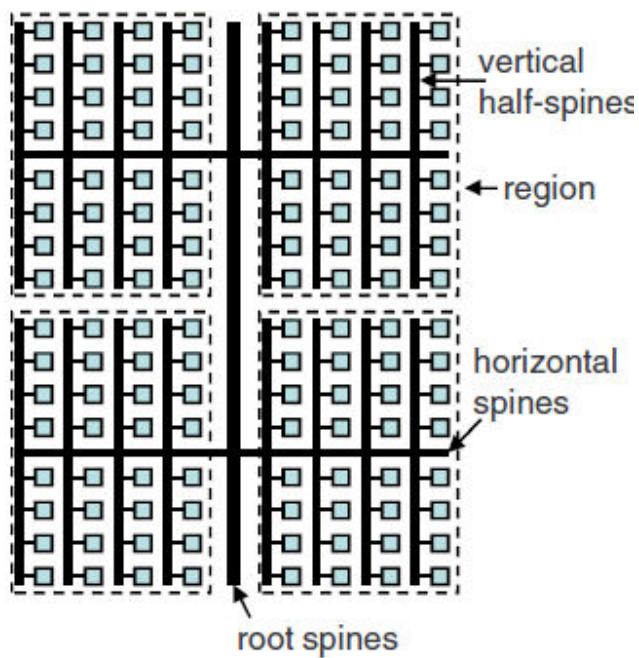
The potential for clock gating as an FPGA power reduction strategy is underscored by the recently announced Xilinx Virtex-6 40 nm FPGA, which permits gating at some clock network switches. Research on FPGA architecture goes hand-in-hand with research on computer-aided design (CAD) tools, as proper architectural evaluation demands CAD tools that leverage architectural enhancements. We present a placement approach whose aim is two-fold: 1)

reducing routing capacitance on the clock network, and 2) placing design objects in a manner that takes advantage of the proposed clock gating architectures. Results show the effectiveness of our placement approach and demonstrate that considerable power reductions are possible with gating, depending on the granularity at which it is applied. The remainder of the paper is organized as follows: Section 2 describes related work on clock power reduction in FPGAs and clock gating. Section 3 introduces the proposed gating architectures and describes our placement approach. An experimental study is presented in Section 4.

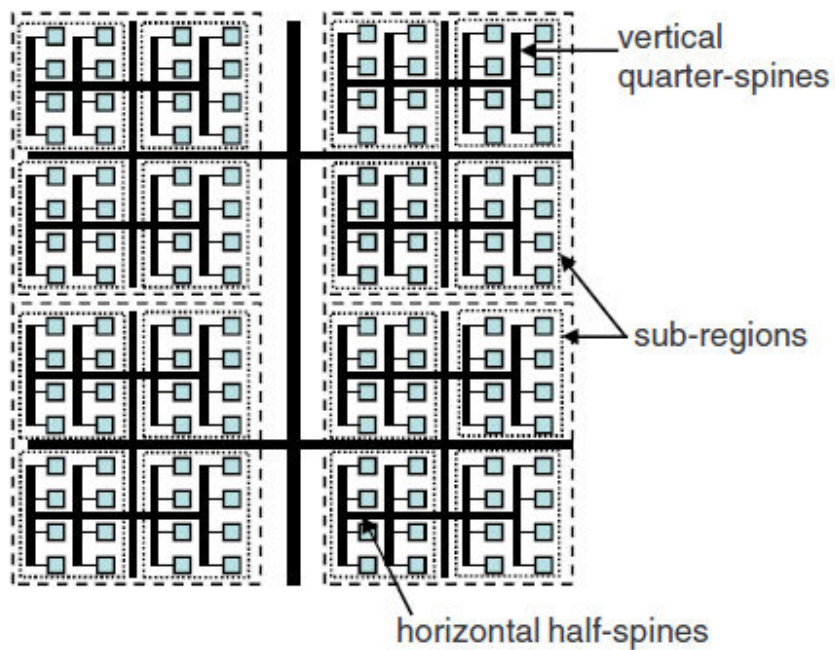
2. CLOCK GATING ARCHITECTURE

Clock signals in FPGAs are distributed through a programmable clock tree. A clock tree resembling that in the Xilinx Virtex-5 FPGA . The device is partitioned into clock regions (4 regions are shown) such that a limited number of clock signals can be delivered into each region. At the top level of the tree are the root spines, which deliver clock signals into regions. Root spines can connect to “horizontal spines” that span each region. Horizontal spines can connect to vertical “half-spines” that span half a region vertically. It is the vertical half-spines that ultimately route the clock signal to clock pins on logic blocks. Routing switches are present where the different spines intersect with one another and such switches are *only* closed on an as-needed basis. A clock signal would not be routed into a region containing no loads requiring the signal, as doing so would needlessly consume power. Fig. shows an alternative “fine-grained” clock architecture resembling that in the Altera Stratix-III FPGA .

In this case, the tree is deeper and there are 16 sub-regions, with 4 sub-regions in each of the quadrant regions. The subregions have the same structure as the quadrant regions in part (a) of the figure. Within sub-regions, horizontal “halfspines” can connect to vertical “quarter-spines” that route clocks to logic blocks. For both architectures in Fig. 1 the number of spines is monotonically decreasing with each level. For example, in the Virtex-5 FPGA, there are 32 root spines at the top level, however, there are only 10 horizontal spines in each region. The structured nature of the clock network and the switches between clock tree levels implies that placement of clock loads has a considerable impact on clock wire usage. Clock load placement can be done such that clock capacitance is minimized, reducing dynamic power. Lamoureux and Wilton were the first to propose clock-aware placement techniques that reduce the number of regions spanned by clock signals and reduce the resources used within regions . Since then, both Actel and Xilinx have published placement approaches for reducing clock spine usage for their respective architectures. Our placer borrows ideas from these prior works and builds upon them to explore clock gating architectures.



a) Coarse-grained architecture



b) Fine-grained architecture

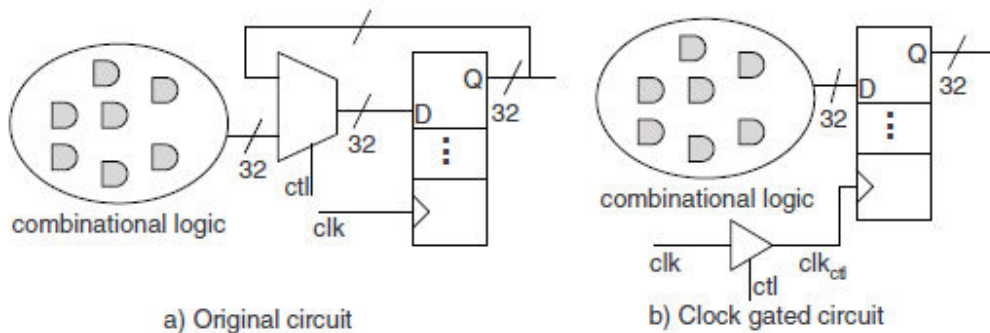


Fig. 2. Clock gating example.

2.1 CLOCK GATING

A combinational logic circuit feeding a 32-bit register file through a multiplexer. The values in the register file are *only* updated with the combinational output when the multiplexer select signal *ctl* is asserted; otherwise, the register file content remains unchanged. Fig. 2(b) shows the circuit after clock gating is applied. The multiplexer is removed

and the clock signal itself is gated with the *ctl* signal, producing a gated clock signal, *clkctl*. Let *actl* represent the percentage of time that *ctl* is asserted on average, ranging from 0 to 1. The frequency of *clkctl* is *actl* · *fclock*, where *fclock* is the frequency of the original clock signal. Power savings are realized through reducing switching activity on a portion

of the clock network. With *ctl* deasserted, the clock signal no longer drives the capacitive load of the 32 register file clock pins. Relative to the case in Fig. 2, clock gating strategies can be considerably more sophisticated in real circuits and generally involve analysis of when registers may be kept idle and subsequent synthesis of clock gating (enable) signals. In this paper, we use the term *enable domain* to refer

to the loads of a clock network that share a common clock

enable signal. Modern commercial FPGA architectures have some capability for clock gating. Stratix-III and Virtex-5 FPGAs

have clock enable pins on the flip-flops in logic blocks; however,

use of such pins does not provide switching activity reduction

on the clock signal, and gives no power savings on

the clock network [11, 10]. Aside from clock enable pins

on flip-flops and other blocks, FPGAs provide gating at the

top-level of the clock tree (as exploited in [8]). The top-level gating shuts down the *entire* clock network and therefore,

cannot be applied when some clock loads are used in ungated

form and some are used in gated form, or when there

exist multiple *enable domains* for a single clock signal. The

recently introduced Xilinx Virtex-6 FPGA has clock gating

capability on a regional basis [9] and Xilinx suggests that

gating can save 30-80% of the clock tree power in some designs

[15]. It is worth noting that use of clock gating is not

limited to general LUT-based logic blocks; it also applies to

the large IP blocks present in modern FPGAs, such as DSP blocks and multipliers.

Our goal is not to synthesize enable gating signals for clocks. Rather, in this work, such enable signals are already present in designs. Our goal is to study the power implications of different hardware architectures for implementing clock gating.

3. GATING ARCHITECTURE AND PLACEMENT

3.1 INTRODUCTION

We envision that clock gating can be implemented naturally and with low-cost using the existing FPGA clock tree architecture. Traditionally, switches in the clock tree network are controlled by SRAM configuration cells. We propose to implement clock gating through a minor hardware modification whereby a subset of clock network switches can also be controlled by an enable signal. The enable signal would be received from general purpose interconnect. We do not believe adding an enable pin onto a fraction of clock tree switches will present an onerous routing demand. In fact, we believe that implementing clock gating through the clock network will lower routing demand vs. implementing gating through clock enable pins on flip-flops in logic blocks, as is often done today.

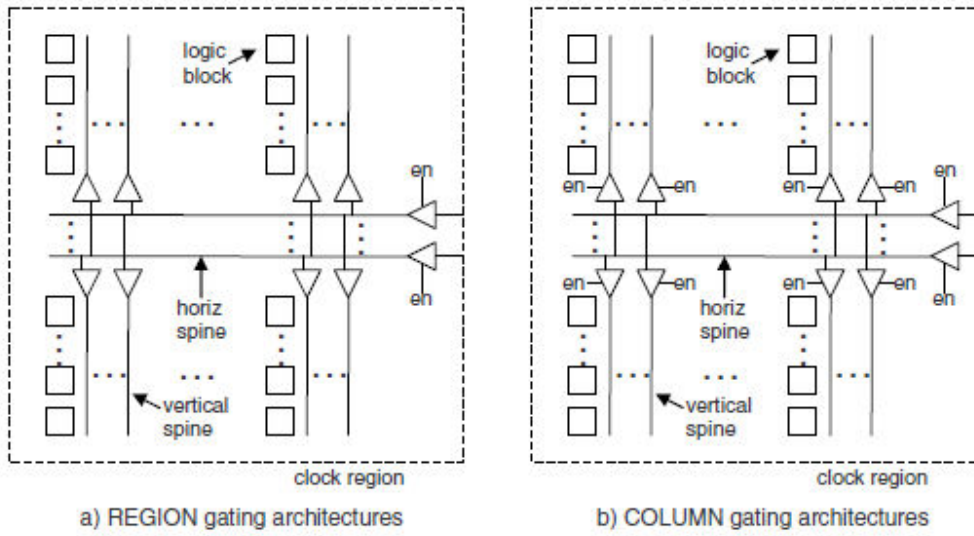
Table 1 shows five different clock gating architectures considered in this paper. The first gating option, NONE, is to have no gating at all, in which case, clock enables must be implemented using clock enable pins on logic blocks.

The next two options, beginning with CG, pertain to the coarse-grained architecture in Fig. 1(a). CG REGION permits gating when clock signals enter a region. That is, the switches in the clock network that connect from the root spine to the horizontal region spines have enable pins on them. CG COLUMN represents a finer granularity of clock gating capability, wherein gating is possible *within* regions where the horizontal spines connect onto the vertical halfspines. The next two options, beginning with FG, pertain to the fine-grained architecture in Fig. 1(b). The gating options are analogous: FG REGION permits gating where clock signals enter sub-regions; FG COLUMN permits gating where the horizontal half-spines in sub-regions connect to vertical quarter-spines. The architectures are illustrated in Fig. 3.

Fig. 3(a) shows the REGION architecture where enables are present on switches entering a region. Fig. 3(b) shows the more flexible COLUMN architecture where enables are also present on switches driving vertical spines in logic block columns. Thus, we consider a broad range of clock gating architectures with various levels of granularity, within clock distribution frameworks that resemble those in commercial chips.

For this work, we assume that each level of wire on the clock network tree can be driven by all wires in the preceding level, as shown in Fig. 4(a). This allows us to create multiple gated forms of a single clock at the next lower level of the tree, as illustrated in Fig. 4(b). This property is useful when there exist multiple enable domains for a single

clock signal. In practice, we believe the same results can be achieved with substantially depopulated switches. Clock networks in commercial chips likely have partially depopulated switches. In the Xilinx Virtex-5, since any 10 of 32 clocks can be driven into any region, there must exist considerable routing flexibility between the top and regional levels. On the other hand, we expect there is less flexibility in the switches within a region, from the horizontal spines to the vertical spines. Note that our flexibility assumptions are not equivalent to a full crossbar, as the clock tree is unidirectional from root to leaves.

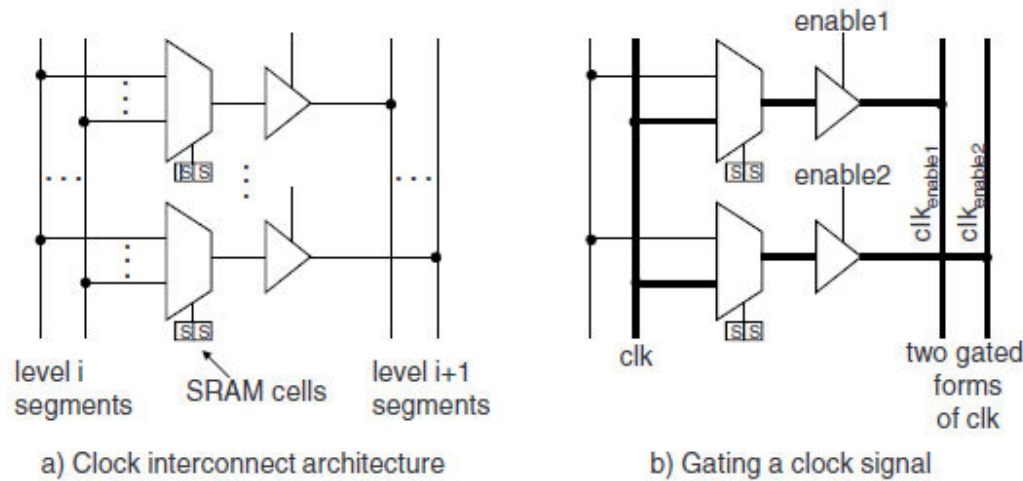


3.2 PLACEMENT

We use a modified version of the VPR placer [16] to explore clock gating architectures. VPR's placer algorithm uses simulated annealing to optimize a composite cost function comprising wirelength and timing performance. We augmented the cost function to incorporate clock network power:

$$Cost = \rho \cdot WL + \sigma \cdot T + v \cdot CP \quad (1)$$

where WL , T and CP represent wirelength, timing and clock power, respectively, and the other parameters are scalar



tuning weights reflecting the importance of each term. For a given placement, we can compute the clock power term in (1) as the sum of the power consumed by each clock signal in the design: $CP = \sum_{clk \in \text{clks}} CP_{clk}$. The power for a particular clock signal clk , CP_{clk} , can be computed based on the current placement and the properties of the FPGA clock network architecture being evaluated. The structured nature of the clock network allows us to rapidly compute a clock routing during placement, and thereby estimate clock capacitance and power. Computing CP_{clk} is more involved than is readily apparent.

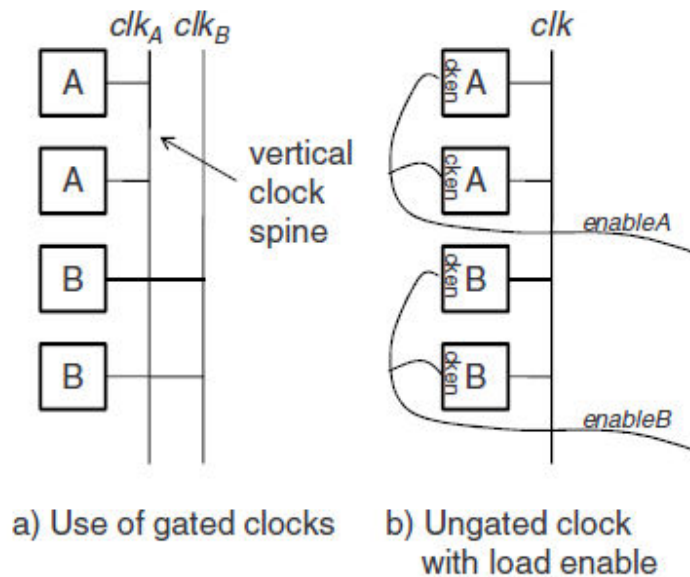
For clock loads within an enable domain of clk , there is a choice: 1) use gated clock signals created via clock enables on switches within the clock distribution network, or 2) use an ungated clock signal along with clock enable pins on the load blocks themselves. It is not always beneficial to use the enables within the clock network (option #1). The decision taken is the one that yields lower power consumption.

To see this, consider the column placement shown in Fig. 5(a) comprising four logic blocks requiring two enable domains of clk , A and B . Option #1 uses gated clocks and is shown in Fig. 5(a). Option #2 uses an ungated clock and enable pins on the blocks themselves and is shown in Fig. 5(b). If C_v is the capacitance of a vertical spine, the clock power consumption of Fig 5(a) is $f_{clk} \cdot \alpha_A \cdot C_v \cdot V^2 + f_{clk} \cdot \alpha_B \cdot C_v \cdot V^2$ Watts, whereas the clock power consumption of Fig. 5(b) is

$f_{clk} \cdot C_v \cdot V^2$ Watts.

Comparing these power values, option #1 should be chosen only if $\alpha_A + \alpha_B < 1$; otherwise, we should use an ungated clock. By extension, if a column contains a placed load that uses the ungated form of clk , then all loads in the column, including those in the gated domains, should use the ungated clock to minimize overall power.

Fig. 6 gives a generalized core procedure we apply in the process of computing $CPclk$. For a clock signal clk in region R , the procedure computes a set of columns U in R that will be fed by clk in ungated form, using a single vertical spine per column [as in Fig. 5(b)]. Parameter COL represents the set of columns in R that have a load of clk placed on them (including both gated and ungated loads). Line 2 initializes U to contain the columns of R where ungated loads of clk are placed. These columns *must* receive clk in ungated form.



4. XILINX SOFTWARE

4.1 INTRODUCTION

Xilinx ISE^[2] (Integrated Synthesis Environment)^[3] is a software tool produced by [Xilinx](#) for synthesis and analysis of [HDL](#) designs, enabling the developer to [synthesize](#) ("compile") their designs, perform [timing analysis](#), examine [RTL](#) diagrams, simulate a design's reaction to different stimuli, and configure the target device with the [programmer](#).

Xilinx ISE is a design environment for FPGA products from Xilinx, and is tightly-coupled to the architecture of such chips, and cannot be used with FPGA products from other vendors.^[3] The Xilinx ISE is primarily used for circuit synthesis and design, while ISIM or the [ModelSim](#) logic simulator is used for system-level testing.^{[4][5]} Other components shipped with the Xilinx ISE include the Embedded Development Kit (EDK), a Software Development Kit (SDK) and ChipScope Pro.^[6]

Since 2012, Xilinx ISE has been discontinued in favor of [Vivado Design Suite](#), that serves the same roles as ISE with additional features for [system on a chip](#) development.^{[7][8]} Xilinx released the last version of ISE in October 2013 (version 14.7), and states that "ISE has moved into the sustaining phase of its product life cycle, and there are no more planned ISE releases."

4.2 USER INTERFACE

The primary user interface of the ISE is the Project Navigator, which includes the design hierarchy (Sources), a [source code editor](#) (Workplace), an output console (Transcript), and a processes tree (Processes).^{[3][9]}

The Design hierarchy consists of design files (modules), whose dependencies are interpreted by the ISE and displayed as a [tree structure](#).^[3] For single-chip designs there may be one main module, with other modules included by the main module, similar to the `main()` subroutine in [C++](#) programs.^[3] Design constraints are specified in modules, which include pin configuration and mapping.^[3]

The Processes hierarchy describes the operations that the ISE will perform on the currently active module.^[3] The hierarchy includes compilation functions, their dependency functions, and other utilities.^[3] The window also denotes issues or errors that arise with each function.^[3]

The Transcript window provides status of currently running operations, and informs engineers on design issues.^[3] Such issues may be filtered to show Warnings, Errors, or both.

4.3 SIMULATION

System-level testing may be performed with ISIM or the [ModelSim](#) logic simulator, and such test programs must also be written in HDL languages.^[3] Test bench programs may include simulated input signal waveforms, or monitors which observe and verify the outputs of the [device under test](#).^[3]

[ModelSim](#) or ISIM may be used to perform the following types of simulations:^[4]

- Logical verification, to ensure the module produces expected results
- Behavioural verification, to verify logical and timing issues
- Post-place & route simulation, to verify behaviour after placement of the module within the reconfigurable logic of the FPGA.
- Xilinx's patented algorithms for synthesis allow designs to run up to 30% faster than competing programs, and allows greater logic density which reduces project time and costs.^[10]
- Also, due to the increasing complexity of FPGA fabric, including memory blocks and I/O blocks, more complex synthesis algorithms were developed that separate unrelated modules into *slices*, reducing post-placement errors.^[10]
- IP Cores are offered by Xilinx and other third-party vendors, to implement system-level functions such as [digital signal processing](#) (DSP), bus interfaces, networking protocols, [image processing](#), [embedded processors](#), and peripherals.^[10] Xilinx has been instrumental in shifting designs from ASIC-based implementation to FPGA-based implementation.

5 FPGA

5.1 INTRODUCTION

A **field-programmable gate array (FPGA)** is an **integrated circuit** designed to be configured by a customer or a designer after manufacturing – hence the term "**field-programmable**". The FPGA configuration is generally specified using a **hardware description language (HDL)**, similar to that used for an **application-specific integrated circuit (ASIC)**. **Circuit diagrams** were previously used to specify the configuration, but this is increasingly rare due to the advent of **electronic design automation** tools.



A Spartan FPGA from [Xilinx](#)

FPGAs contain an array of **programmable logic blocks**, and a hierarchy of reconfigurable interconnects that allow the blocks to be "wired together", like many logic gates that can be interwired in different configurations. **Logic blocks** can be configured to perform complex **combinational functions**, or merely simple **logic gates** like **AND** and **XOR**. In most FPGAs, logic blocks also include **memory elements**, which may be simple **flip-flops** or more complete blocks of memory.^[1] Many FPGAs can be reprogrammed to implement different **logic functions**,^[2] allowing flexible **reconfigurable computing** as performed in **computer software**.

5.2 TECHNICAL DESIGN

Contemporary field-programmable gate arrays (FPGAs) have large resources of [logic gates](#) and RAM blocks to implement complex digital computations.^[2] As FPGA designs employ very fast I/O rates and bidirectional data [buses](#), it becomes a challenge to verify correct timing of valid data within setup time and hold time.

[Floor planning](#) enables resource allocation within FPGAs to meet these time constraints. FPGAs can be used to implement any logical function that an [ASIC](#) can perform. The ability to update the functionality after shipping, [partial re-configuration](#) of a portion of the design^[3] and the low non-recurring engineering costs relative to an ASIC design (notwithstanding the generally higher unit cost), offer advantages for many applications.^[1]

Some FPGAs have analog features in addition to digital functions. The most common analog feature is a programmable [slew rate](#) on each output pin, allowing the engineer to set low rates on lightly loaded pins that would otherwise [ring](#) or [couple](#) unacceptably, and to set higher rates on heavily loaded pins on high-speed channels that would otherwise run too slowly.^{[4][5]} Also common are [quartz-crystal oscillators](#), on-chip resistance-capacitance oscillators, and [phase-locked loops](#) with embedded [voltage-controlled oscillators](#) used for clock generation and management and for high-speed serializer-deserializer (SERDES) transmit clocks and receiver clock recovery. Fairly common are differential [comparators](#) on input pins designed to be connected to [differential signaling](#) channels. A few "[mixed signal FPGAs](#)" have integrated peripheral [analog-to-digital converters](#) (ADCs) and [digital-to-analog converters](#) (DACs) with analog signal conditioning blocks allowing them to operate as a [system-on-a-chip](#) (SoC).^[6] Such devices blur the line between an FPGA, which carries digital ones and zeros on its internal programmable interconnect fabric, and [field-programmable analog array](#) (FPAA), which carries analog values on its internal programmable interconnect fabric.

5.2 HISTORY

The FPGA industry sprouted from [programmable read-only memory](#) (PROM) and [programmable logic devices](#) (PLDs). PROMs and PLDs both had the option of being programmed in batches in a factory or in the field (field-programmable). However, programmable logic was hard-wired between logic gates.^[7]

[Altera](#) was founded in 1983 and delivered the industry's first reprogrammable logic device in 1984 – the EP300 – which featured a quartz window in the package that allowed users to shine an ultra-violet lamp on the die to erase the [EPROM](#) cells that held the device configuration.^[8] In December 2015, [Intel](#) acquired Altera.

[Xilinx](#) co-founders [Ross Freeman](#) and [Bernard Vonderschmitt](#) invented the first commercially viable field-programmable [gate array](#) in 1985 – the XC2064.^[9] The XC2064 had programmable gates and programmable interconnects between gates, the beginnings of a new technology and market.^[10] The XC2064 had 64 configurable logic blocks (CLBs), with two three-input [lookup](#)

tables (LUTs).^[11] More than 20 years later, Freeman was entered into the [National Inventors Hall of Fame](#) for his invention.^{[12][13]}

In 1987, the [Naval Surface Warfare Center](#) funded an experiment proposed by Steve Casselman to develop a computer that would implement 600,000 reprogrammable gates. Casselman was successful and a patent related to the system was issued in 1992.^[7]

Altera and Xilinx continued unchallenged and quickly grew from 1985 to the mid-1990s, when competitors sprouted up, eroding significant market share. By 1993, Actel (now [Microsemi](#)) was serving about 18 percent of the market.^[10] By 2013, Altera (31 percent), Actel (10 percent) and Xilinx (36 percent) together represented approximately 77 percent of the FPGA market.^[14]

The 1990s were a period of rapid growth for FPGAs, both in circuit sophistication and the volume of production. In the early 1990s, FPGAs were primarily used in [telecommunications](#) and [networking](#). By the end of the decade, FPGAs found their way into consumer, automotive, and industrial applications.

5.3 APPLICATION

An FPGA can be used to solve any problem which is [computable](#). This is trivially proven by the fact that FPGAs can be used to implement a [soft microprocessor](#), such as the Xilinx [MicroBlaze](#) or Altera [Nios II](#). Their advantage lies in that they are significantly faster for some applications because of their [parallel nature](#) and [optimality](#) in terms of the number of gates used for certain processes.^[34]

FPGAs originally began as competitors to [CPLDs](#) to implement [glue logic](#) for [printed circuit boards](#). As their size, capabilities, and speed increased, FPGAs took over additional functions to the point where some are now marketed as full [systems on chips](#) (SoCs). Particularly with the introduction of dedicated [multipliers](#) into FPGA architectures in the late 1990s, applications which had traditionally been the sole reserve of [digital signal processor hardware](#) (DSPs) began to incorporate FPGAs instead.^{[35][36]}

Another trend in the use of FPGAs is [hardware acceleration](#), where one can use the FPGA to accelerate certain parts of an algorithm and share part of the computation between the FPGA and a generic processor.^[2] The search engine [Bing](#) is noted for adopting FPGA acceleration for its search algorithm in 2014.^[37] As of 2018, FPGAs are seeing increased use as [AI accelerators](#) including Microsoft's so-termed "Project Catapult"^[19] and for accelerating [artificial neural networks](#) for [machine learning](#) applications.

Traditionally,^[when?] FPGAs have been reserved for specific [vertical applications](#) where the volume of production is small. For these low-volume applications, the premium that companies pay in hardware cost per unit for a programmable chip is more affordable than the development resources spent on creating an ASIC. As of 2017, new cost and performance dynamics have broadened the range of viable applications.

In Italy, the company [Biominesrl](#) has patented the application of FPGA processors, improving its yield in crypto currency mining.

6.Mimas V2 Spartan 6 FPGA Development Board with DDR SDRAM

6.1 INTRODUCTION

MIMAS V2 is a feature packed yet low cost FPGA Development board featuring Xilinx Spartan-6 FPGA. It is specially designed for experimenting and learning system design with FPGAs. This development board features SPARTAN XC6SLX9 CSG324 FPGA with onboard 512Mb DDR SDRAM. The USB 2.0 interface provides fast and easy configuration download to the on-board SPI flash. No need to buy an expensive programmer or special downloader cable to download the bit stream to the board.

Applications

- Product Prototype Development
- Signal Processing
- Learning Digital Electronics
- Educational tool for schools and universities

Board features

- FPGA: Spartan XC6SLX9 in CSG324 package
- DDR: 166MHz 512Mb LPDDR (MT46H32M16LF/W949D6CBHX6E)
- Flash memory: 16 Mb SPI flash memory (M25P16)
- USB 2.0 interface for On-board flash programming
- FPGA configuration via JTAG and USB
- 8 LEDs Six Push Buttons and 8 way DIP switch for user defined purposes
- VGA Connector
- Stereo Jack
- Micro SD Card Adapter
- Three Digit Seven Segment Display.
- 32 IOs for user defined purposes
- Four 6×2 Expansion Connectors
- On-board voltage regulators for single power rail operation

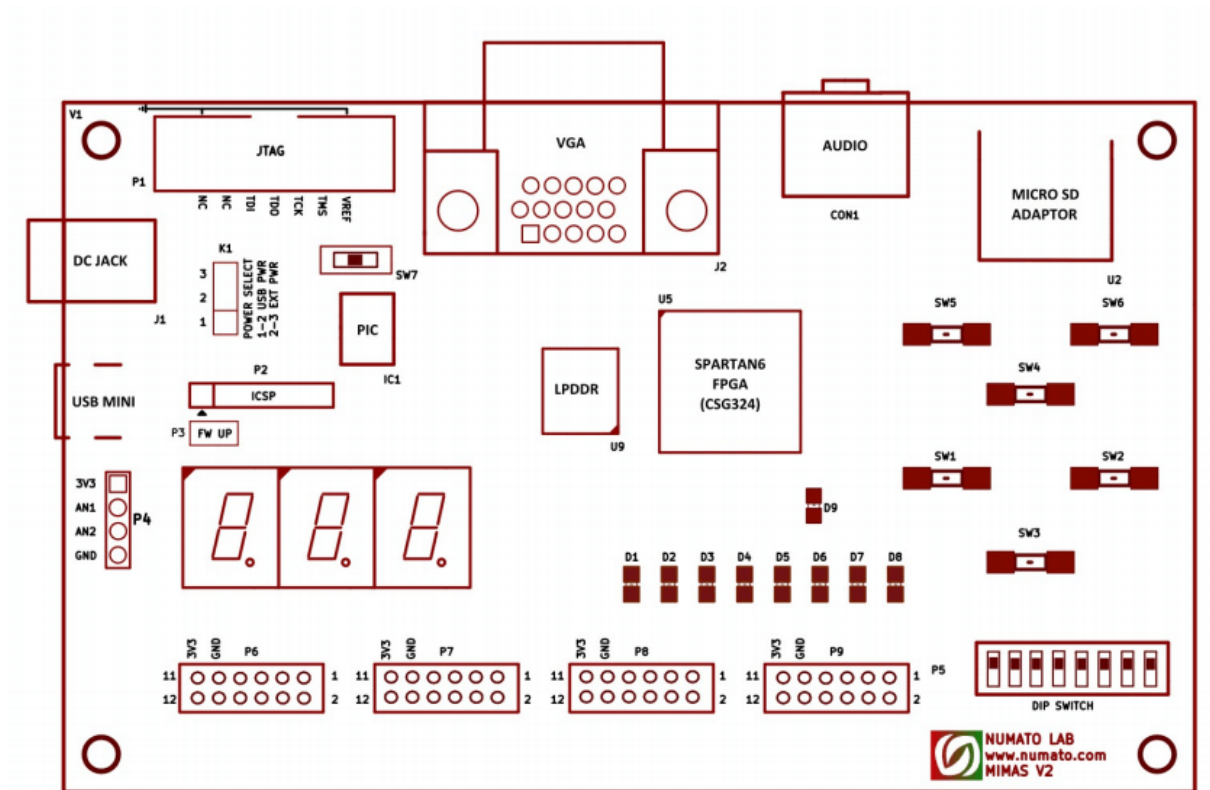
5.2 HOW TO USE MIMAS SPARTANV2 FPGA BOARD

5.2.1 COMPONENTS REQUIRED

Along with the module, you may need the items in the list below for easy and fast installation.

1. USB A to Mini B cable.
2. DC Power supply (Optional).

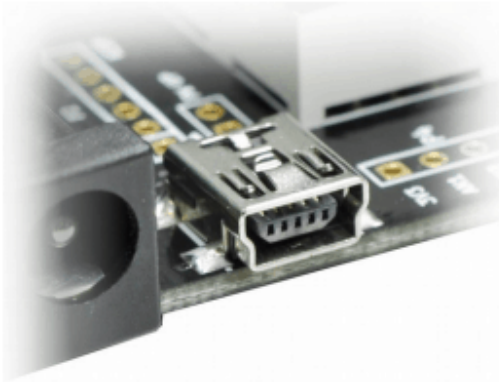
5.2.2 CONNECTION DIAGRAM



This diagram should be used as a reference only. For detailed information, see MIMAS V2 schematics at the end of this documentation. Details of individual connectors are as below.

5.3 USB INTERFACE

The on board full speed USB controller helps a computer to communicate with this module. Use a USB A to Mini B cable to connect with a PC. By default the module is powered from USB so make sure not to overcrowd unpowered USB hubs.



5.4 DC POWER SUPPLY

This module uses +5V power supply to function properly. **By default the board is configured to use +5V supply from USB. So an external +5V power is not required unless USB port is unable to supply enough current. In most cases USB ports are capable of providing enough current for the module. Current requirement for this board largely depends on your application. Please consult FPGA datasheet for more details on power requirements.** If for any reason, an external 5V power supply needs to be used for the module, the Power select jumper should be configured properly before connecting the power supply. Please refer to the marking on the board for more details.

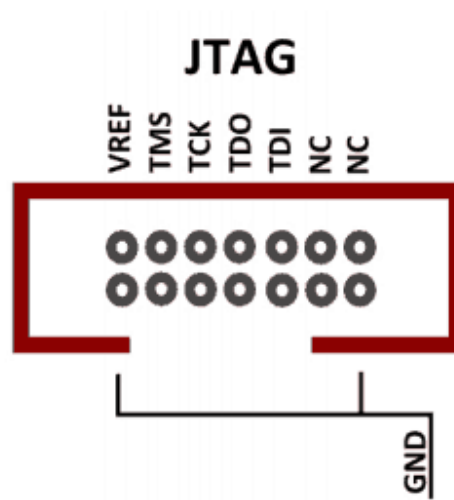


5.5 POWER SELECTOR

The Power Select header K1 is used to configure the power source for the board. The jumper in pin 1 and 2 is shorted to switch the power source to on board USB port and pin 2 and 3 to use the external DC power.

5.6 JTAG CONNECTOR

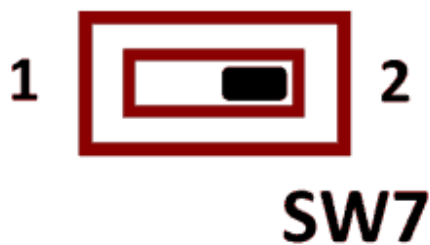
JTAG connector provides access to FPGA's JTAG pins. A XILINX platform cable can be



used for JTAG programming.

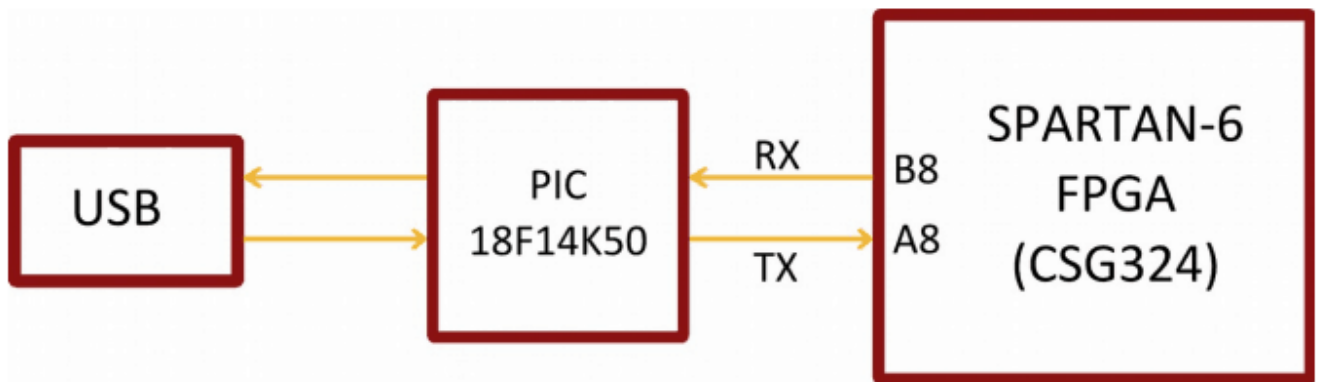
5.7 CONFIGURATION MODE SELECTION

Slide switch SW7 is used to switch between the USB configuration mode and UART. Slide the switch to Position 1 to download bit stream through USB configuration tool and Position 2 to use the interface as a UART in order to communicate from your code in FPGA with the PC. By default the board is shipped with slide switch position in USB configuration tool mode.



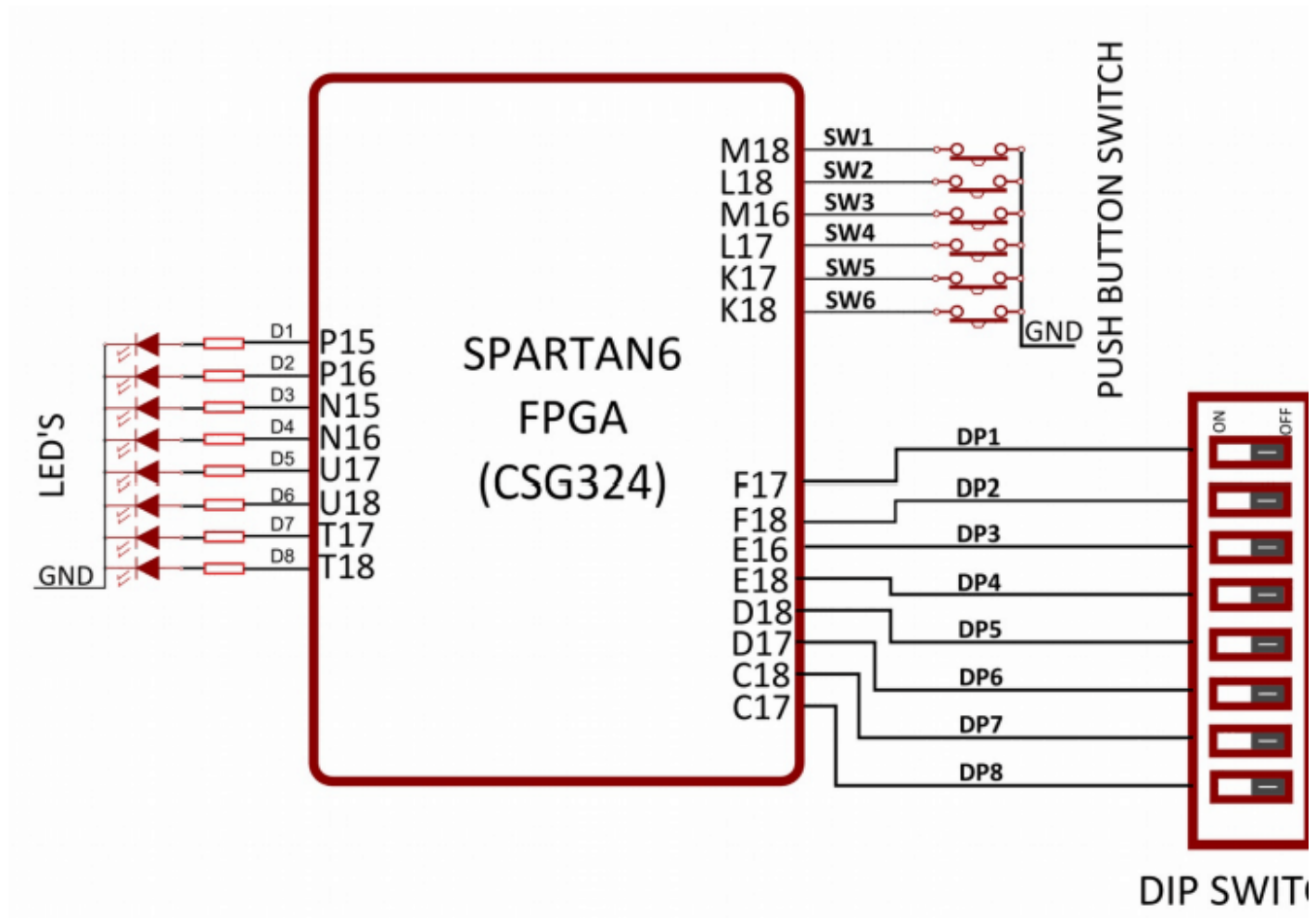
5.8 UART

The MIMAS V2 includes USB-UART, which helps to establish the communication between the code in the FPGA and any application running on the PC. Data can be sent and received from the FPGA by using Serial Terminal at baud rate 19200.



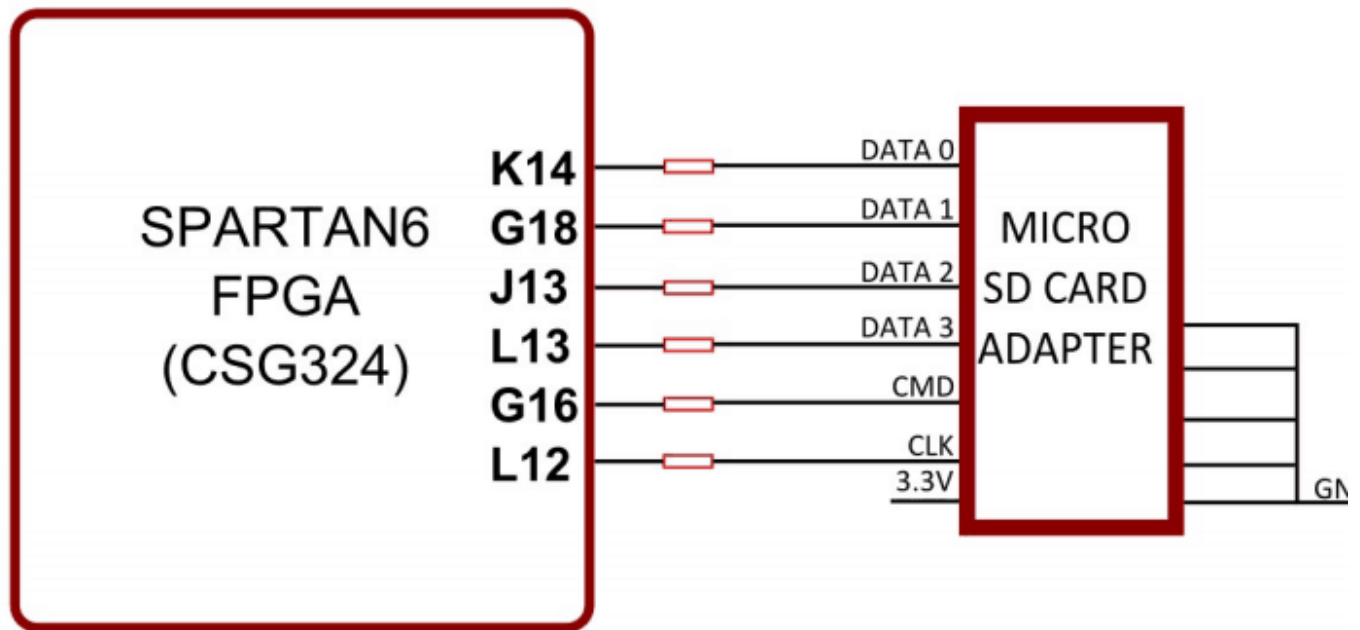
5.9 LED, Push Button and Dip Switch

MIMAS V2 has six push button switches, an eight position DIP switch and eight LEDs for human interaction. All switches are directly connected to Spartan 6 FPGA and can be used in your design with minimal effort.



5.10 MICRO SD

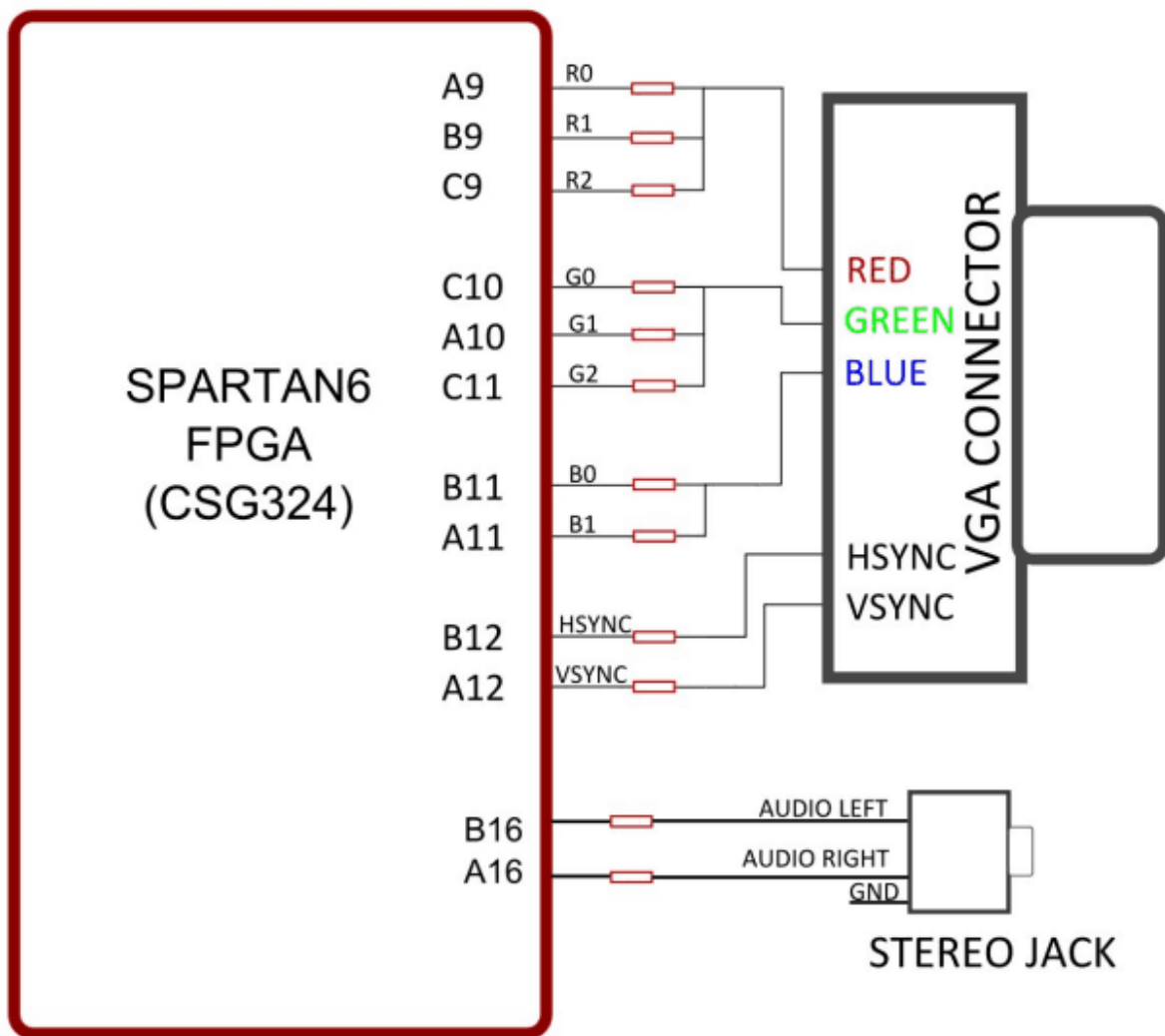
MIMAS V2 features a Micro SD adapter on-board. By installing a Micro SD card, you can add data logging, media storage and other file storage to your design.



5.11 VGA AND AUDIO

The VGA interface provides this board the ability to generate VGA signals from FPGA and display information any Display/monitor that supports standard VGA connector. This VGA interface uses resistor network based DAC for easy code implementation. This 8 bit VGA interface can display up to 256 colors.

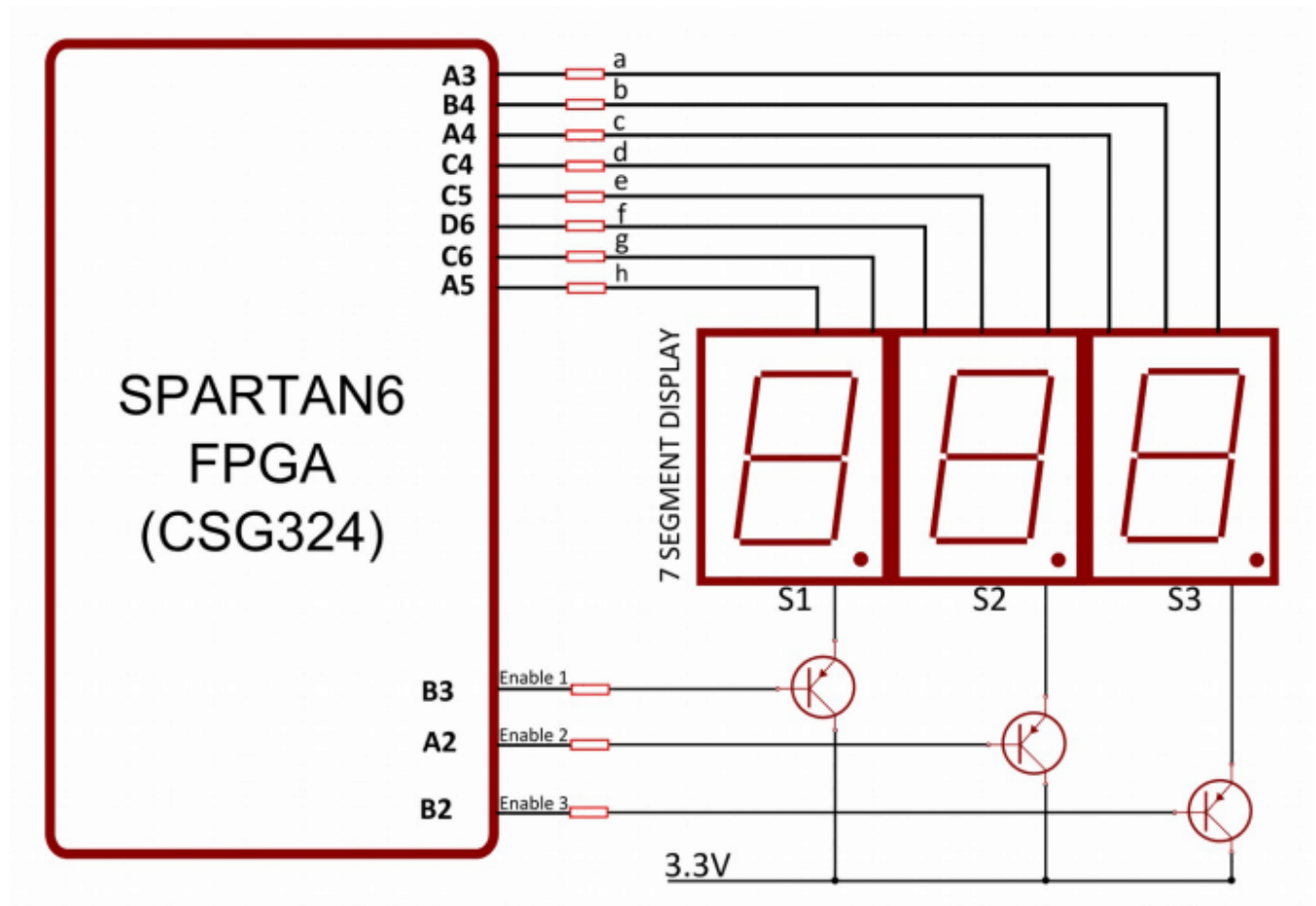
Two IOs on the FPGA are dedicated for generating two channels of audio. Different audio tones can be generated by using PWM and Frequency synthesis.



5.13 SEGMENT LED DISPLAY

This board features three 7-segment LED display multiplexed for low pin count operation. Each module can be separately turned on and off with the three switching transistors.

Note: All signals (*a, b, c, d, e, f, g, dot, enable 1, enable 2, enable 3*) used for controlling 7-Segment display are **active-low** signals. So, for example, for displaying “8” in display-2, users need to drive *Enable 2* to 0 as well as drive signals *a, b, c, d, e, f* to 0. All other signals need to be driven to 1.



5.12 GPIOs

This board is equipped with 32 user IO pins that can be used for various custom applications. Pin assignments on the connectors are available in the tables below.

HEADER P6

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
1	IO_L43P_2	U7
2	IO_L43N_2	V7

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
3	IO_L63P_2	T4
4	IO_L63N_2	V4
5	IO_L49P_D3_2	U5
6	IO_L49N_D4_2	V5
7	IO_L62P_D5_2	R3
8	IO_L62N_D6_2	T3
9	GND	NA
10	GND	NA
11	VCCAUX	NA

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
12	VCCAUX	NA

HEADER P7

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
1	IO_L41P_2	U8
2	IO_L41N_VREF_2	V8
3	IO_L31P_GCLK31_D14_2	R8
4	IO_L31N_GCLK30_D15_2	T8
5	IO_L48P_D7_2	R5
6	IO_L48N_RDWR_B_VREF_2	T5
7	IO_L32P_GCLK29_2	T9

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
----------------	-----------------	----------------------------

8	IO_L32N_GCLK28_2	V9
---	------------------	----

9	GND	NA
---	-----	----

10	GND	NA
----	-----	----

11	VCCAUX	NA
----	--------	----

12	VCCAUX	NA
----	--------	----

HEADER P8

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
----------------	-----------------	----------------------------

1	IO_L16P_2	R11
---	-----------	-----

2	IO_L16N_VREF_2	T11
---	----------------	-----

3	IO_L29P_GCLK3_2	R10
---	-----------------	-----

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
4	IO_L29N_GCLK2_2	T10
5	IO_L14P_D11_2	U13
6	IO_L14N_D12_2	V13
7	IO_L23P_2	U11
8	IO_L23N_2	V11
9	GND	NA
10	GND	NA
11	VCCAUX	NA
12	VCCAUX	NA

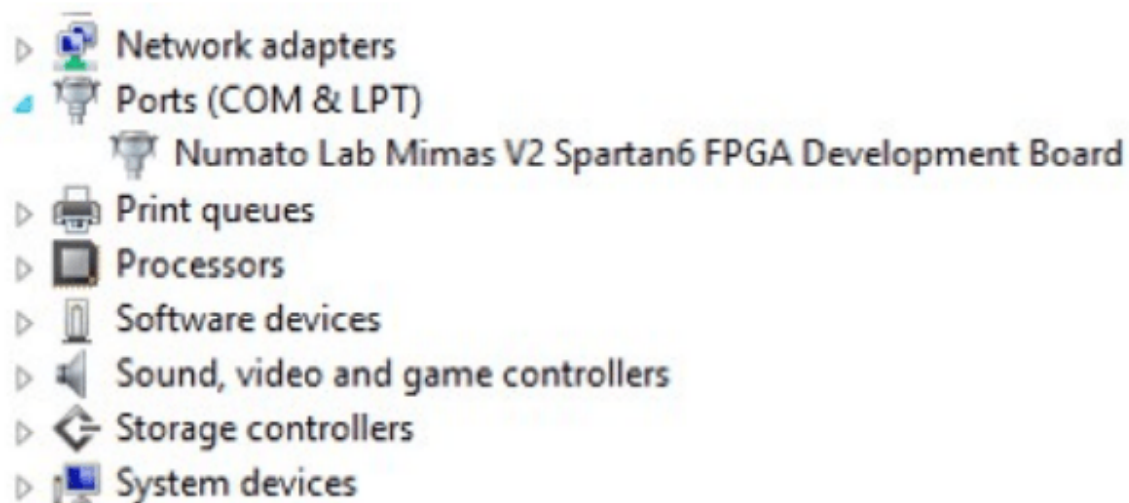
HEADER P9

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
1	IO_L43P_GCLK5_M1DQ4_1	H17
2	IO_L43N_GCLK4_M1DQ5_1	H18
3	IO_L44P_A3_M1DQ6_1	J16
4	IO_L44N_A2_M1DQ7_1	J18
5	IO_L41P_GCLK9_IRDY1_M1RASN_1	K15
6	IO_L41N_GCLK8_M1CASN_1	K16
7	IO_L42P_GCLK7_M1UDM_1	L15
8	IO_L42N_GCLK6_TRDY1_M1LDM_1	L16
9	GND	NA

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
10	GND	NA
11	VCCAUX	NA
12	VCCAUX	NA

6.Driver Installation

6.1.Installing on Windows



This product requires a driver to be installed for proper functioning when used with Windows. The driver package can be downloaded from the product page. To install the driver, unzip the contents of the downloaded driver package to a folder. Attach USB cable to the PC and when asked by Windows device installation wizard, point to the folder where driver files are present. When driver installation is complete, the module should appear in

Windows Device Manager as a serial port (see the picture on the right). Note down the name of the serial port (COM1, COM2 etc..). This information is required while programming the module with configuration tool.

6.2 Installing on Linux

To use this product with Linux, USB CDC driver needs to be compiled in with the kernel. Fortunately, **most Linux distributions (Ubuntu, Redhat, Debian etc.) has this driver pre-installed.** The chances of you requiring to rebuild the kernel to include the USB CDC driver is very slim. When connected to a Linux machine, this product should appear as a serial port in the /dev directory. Usually the name of the device will be “ttyACMx” or similar. The name may be different depending on the Linux distribution you have.

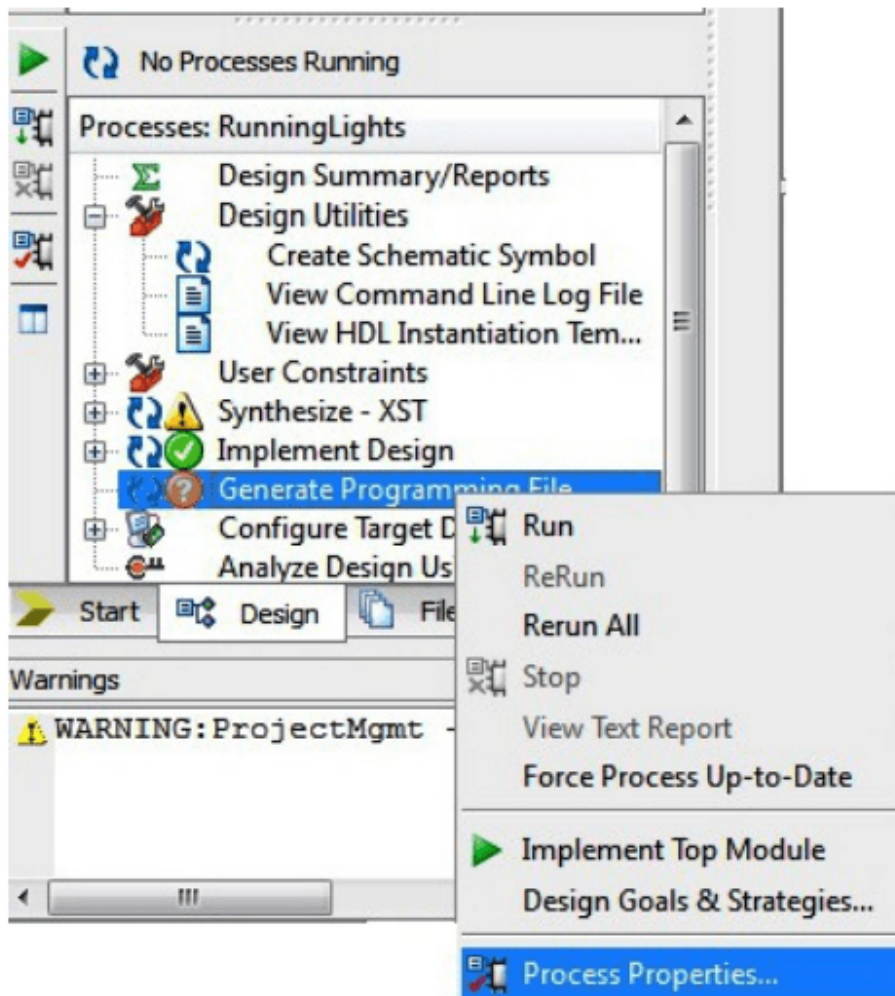
6.3 Installing on Mac

Similar to Linux, Mac operating system comes with the required drivers pre-installed. When connected to a Mac computer, the device should appear as a serial port.

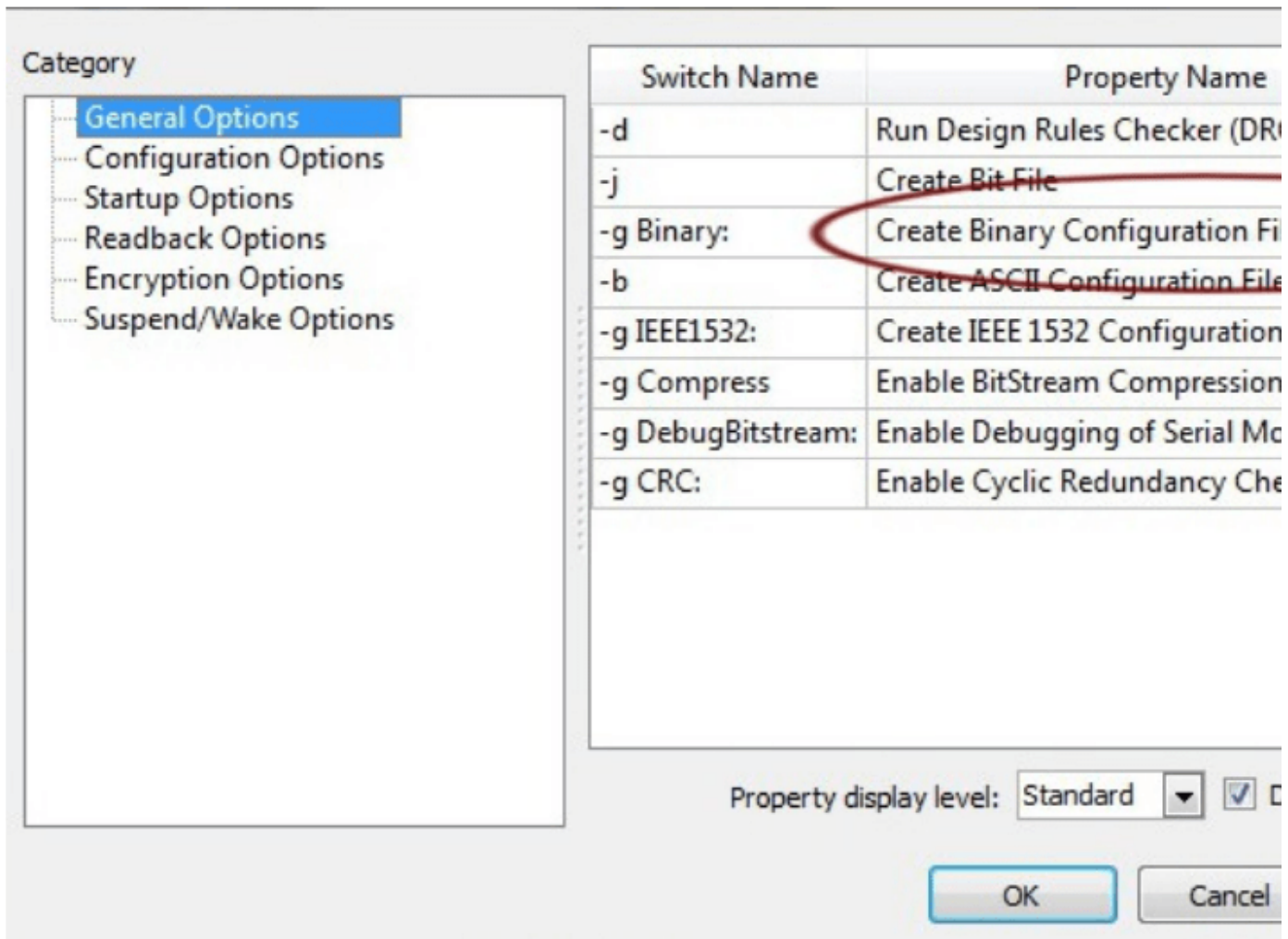
6.4 Generating Bit Stream for MIMAS V2

HDL design needs to be converted to bit stream before it can be programmed to FPGA. MIMAS V2 configuration tool at this time accepts only binary (.bin) bit stream created by XILINX ISE (<http://www.xilinx.com/tools/webpack.htm>). Once the HDL is synthesized, it is easy to create a binary bit stream out of it. Please follow the Steps below to generate binary bit stream from your design using ISE Web Pack.

Step 1: Right click on the “Generate Programming File” option in “Processes” window.



Step 2: Select “Process Properties” from the pop up menu. In the dialog box, check “Create Binary Configuration File” Check box and click “Apply”.



Step 3: Click “OK” to close the dialog box. Right click on “Generate Programming File” option again and select “Run”. Now you will be able to find a .bin file in the project directory and that file can be used for MIMAS V2 configuration.

6.5 Powering Up MIMAS V2

MIMAS V2 can be powered directly from USB port so make sure that you are using a USB port that can power the board properly. It is recommended to connect the board directly to the PC instead using a hub. It is practically very difficult to estimate the power consumption of the board, as it depends heavily on your design and the clock used. XILINX provides tools to estimate the power consumption. In any case if power from USB is not enough for your application, an external supply can be applied to the board. MIMAS V2 requires two

different voltages, a 3.3V and a 1.2V supply. On-board regulators derive these voltages from the USB/Ext power supply

6.6 Configuring MIMAS V2

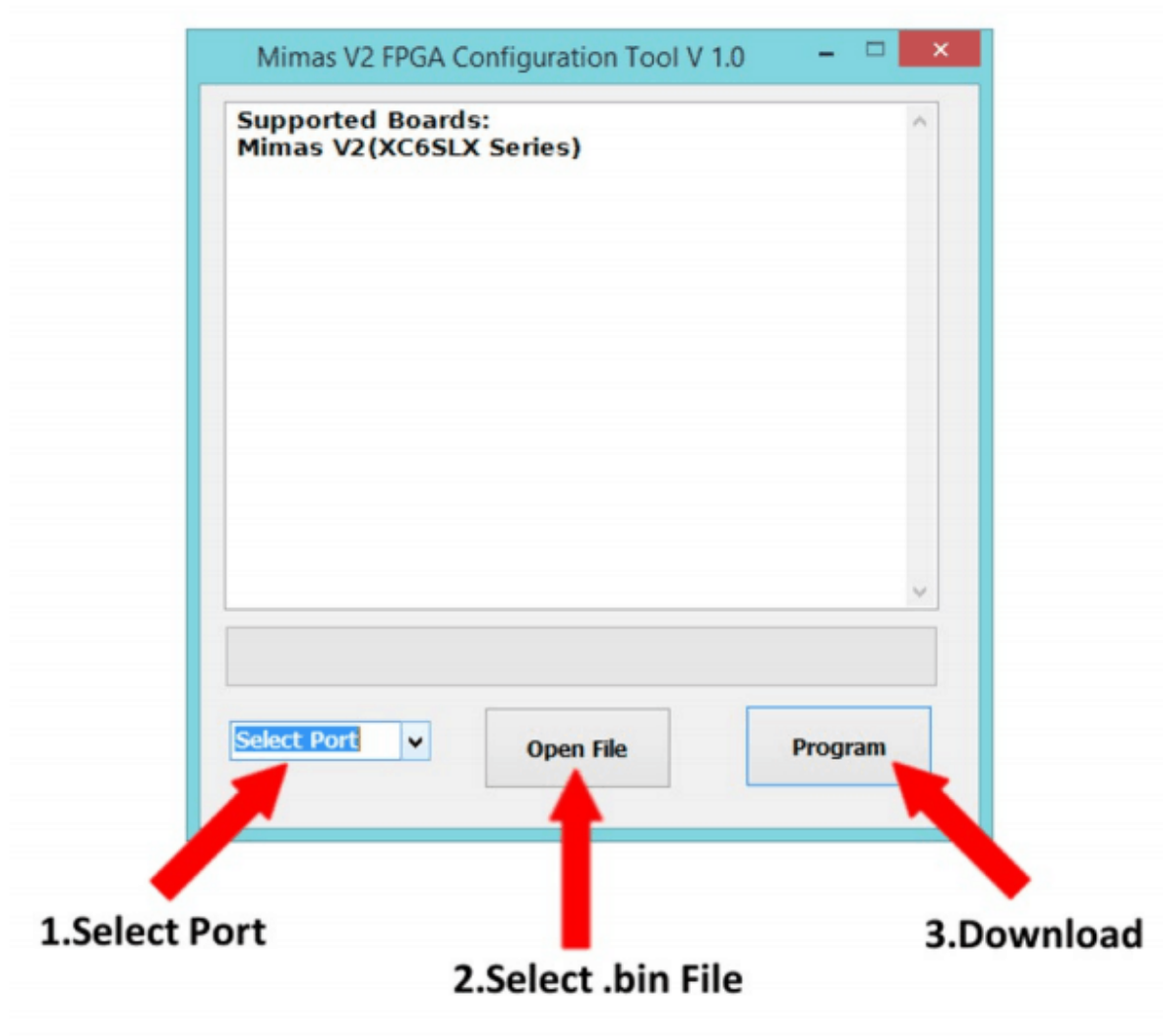
The MIMAS V2 Spartan6 module can be configured by two methods,

- a) Using MIMAS V2 configuration tool through USB.
- b) Using the Xilinx programming cable.

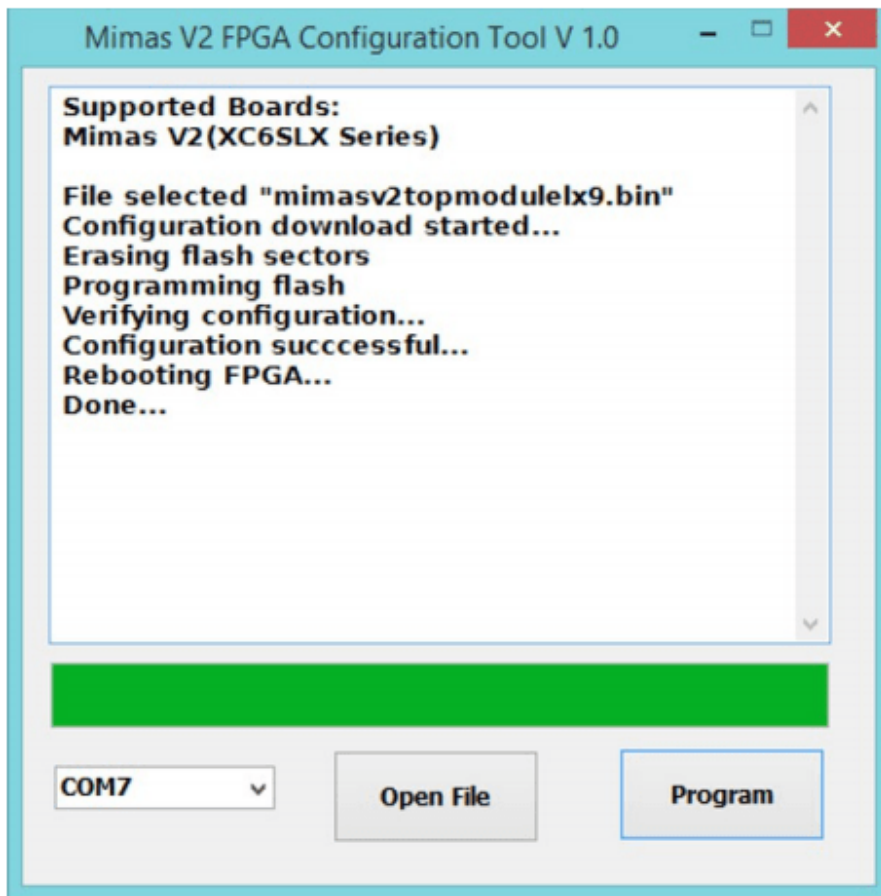
6.7 Configuring MIMAS V2 Using Configuration Tool

MIMAS V2 has an on-board micro-controller which facilitates easy reprogramming of on-board SPI flash through USB interface. The micro-controller receives bit stream from the host application and program it in to the SPI Flash and lets the FPGA boot from the flash. The MIMAS V2 configuration application can be downloaded from www.numato.com for free. When MIMAS V2 is connected to PC, it shows up as a COM port in Device Manager. Run configuration application, select correct COM Port before downloading bit stream. Click on “Open File” to select the bit stream file (.bin) and press “Program” button to download the bit stream. Wait till the download process is finished. Once the download process is over, the configuration controller will try to boot the FPGA from the SPI Flash automatically. Follow the below steps.

Step 1: Make sure you have selected USB configuration mode (Slide SW7 to position 1. Refer to the section “Configuration Mode Selection” for more information). Run MIMAS V2 Configuration Tool and select the correct port (Refer to section “Driver installation” for more information on finding port number). Click Open file button and select the .bin file.



Step 2: Click on “Program” button. Wait till “Done” appears on the screen.

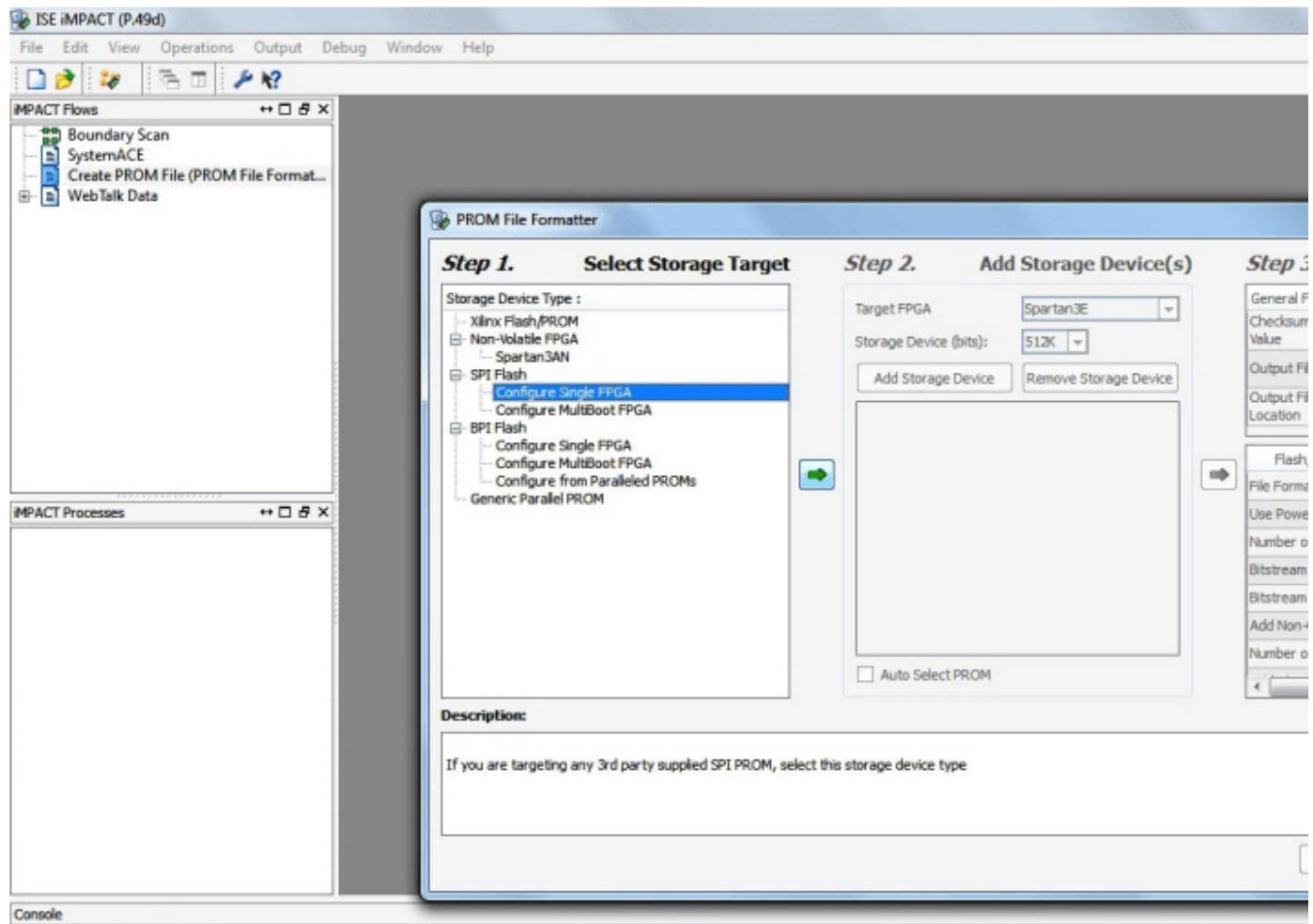


6.8 Configuring MIMAS V2 Using JTAG

MIMAS V2 Spartan6 module features an on-board JTAG connector which facilitates easy reprogramming of SRAM and on-board SPI flash through JTAG programmer like “XILINX Platform-cable usb”. Programming MIMAS V2 using JTAG requires “XILINX ISE iMPACT” software which is bundled with XILINX ISE Design Suite. To program SPI flash we need a “.mcs” file which needs to be generated from the “.bit” file. Steps for generating “.mcs” file is discussed below. Programming FPGA SRAM does not require a mcs file to be generated.

Generating “.mcs” file for MIMAS V2

Step 1: Open ISE iMPACT. Click on “Create PROM file(PROM file formatter)”. In the dialog box, select “Configure Single FPGA” in storage device type. Then click on the green arrow at the right side.



Step 2: Select 16M in Storage Device (bits) list. Now click on “Add Storage Device”, then the green arrow at the right side.

Step 2. Add Storage Device(s)

Storage Device (bits)

Add Storage Device

128K
256K
512K
1M
2M
4M
8M
16M
32M
64M
128M
256M

Auto Select PROM

Step 3: Set an output file name and an output file location (the “.mcs” file will be generated at this location which will be required later for programming the FPGA), then click OK twice, then select the “.bit” file we already generated then click Open and click NO when it prompts to add another device file.

Step 3. **Enter Data**

General File Detail	Value
Checksum Fill Value	FF
Output File Name	MimasV2
Output File Location	C:\Xilinx\14.4\ISE_DS\ 

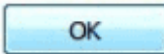
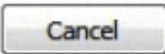
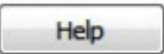


Flash/PROM File Property	Value
File Format	MCS 
Add Non-Configuration Data Files	No 

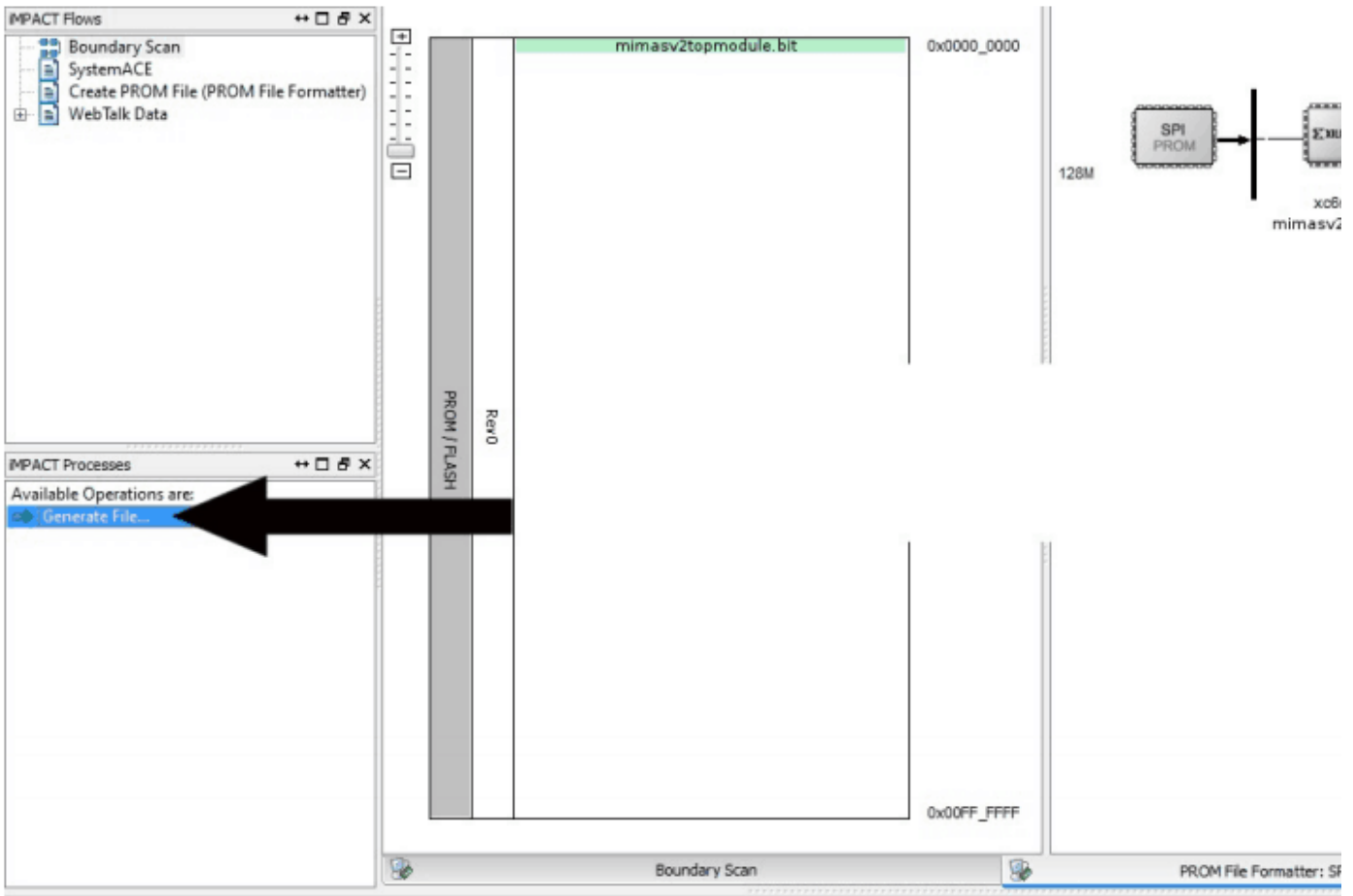
ed mode.

ed to calculate the checksum of the unused portions.

le format your PROM programmer uses, you output a MCS

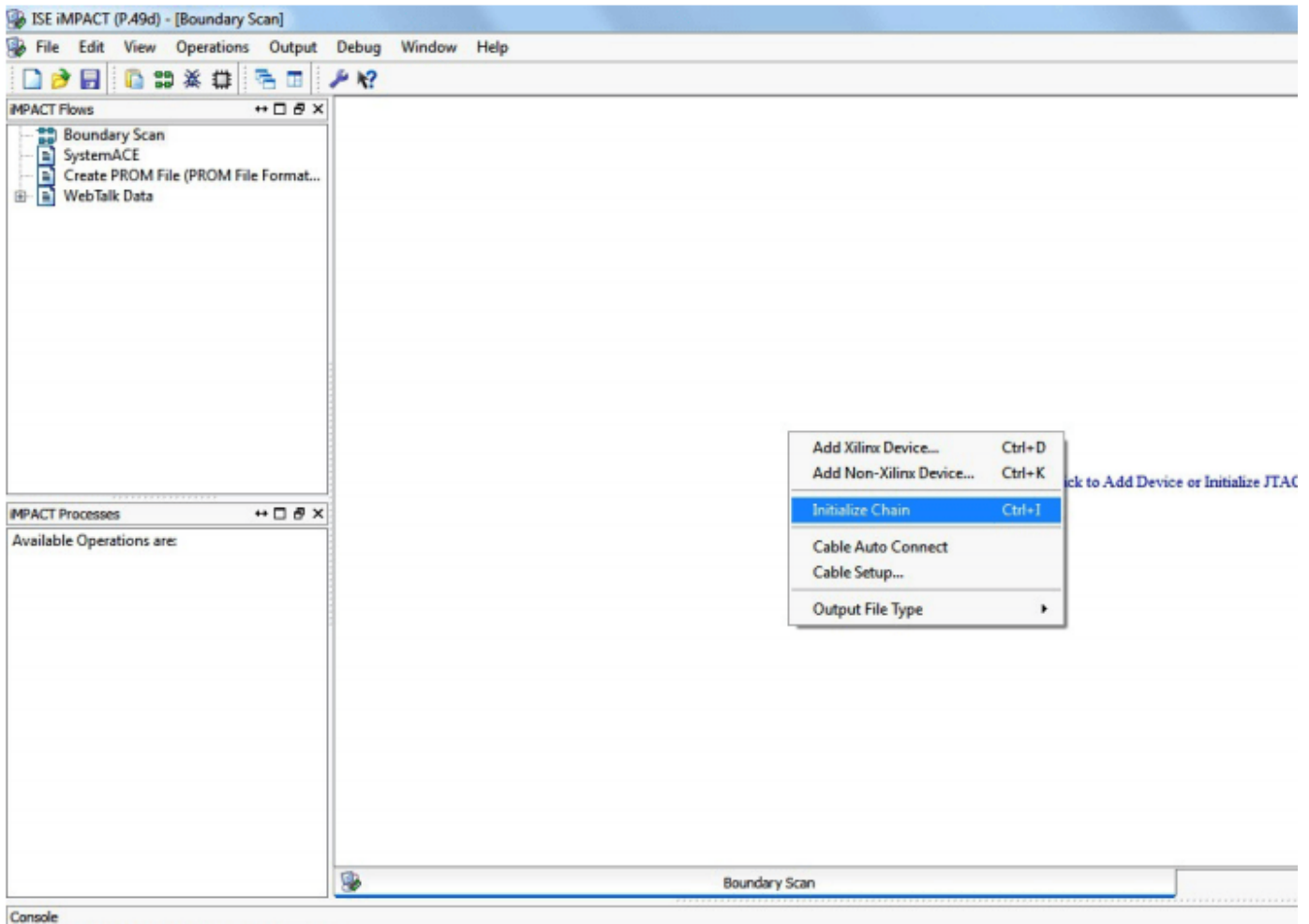
  

Step 4: Double click on “Generate File”. A message “Generate Succeeded” will be displayed as shown in fig below if the mcs file is generated successfully.

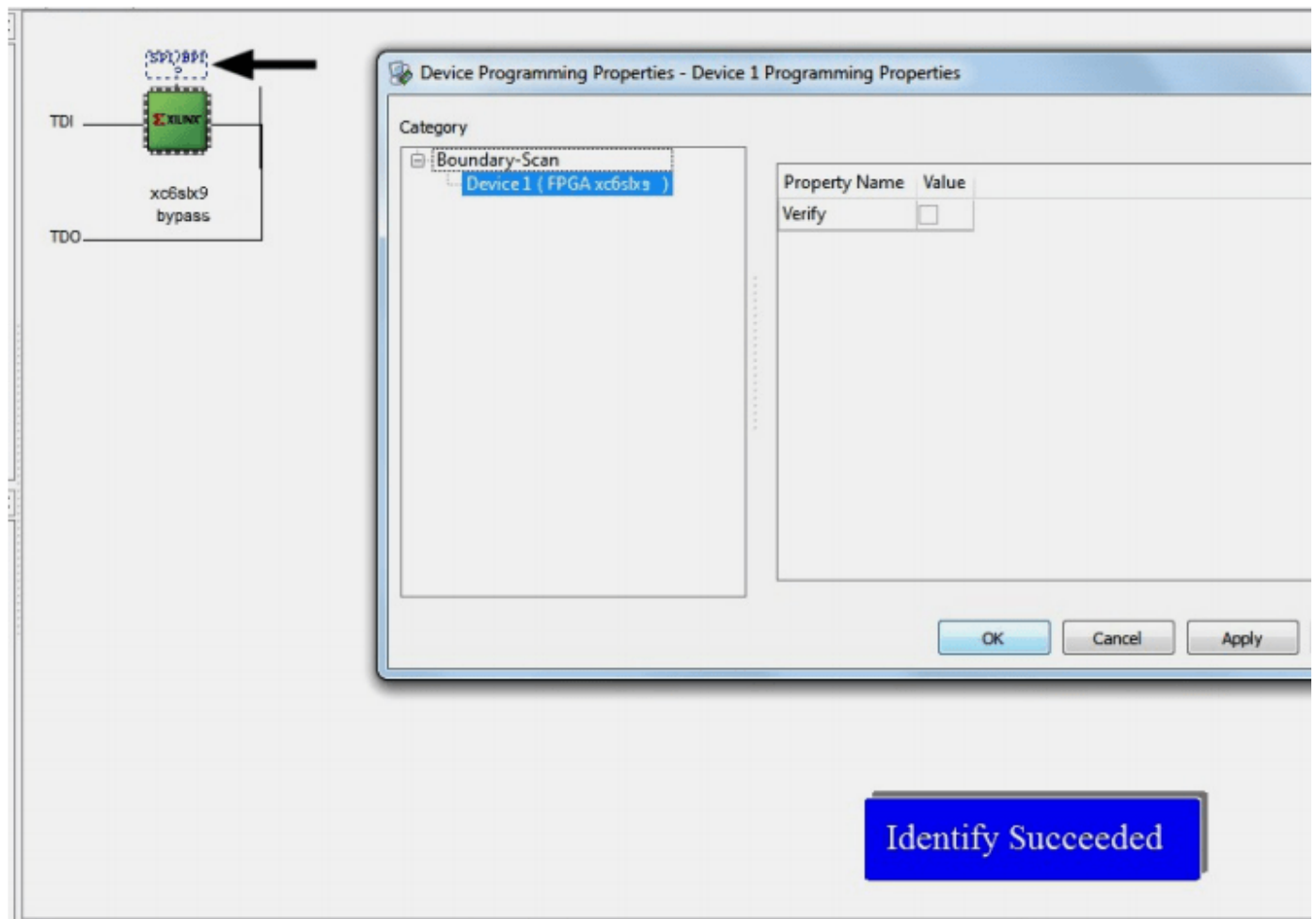


6.9 Programming Onboard SPI flash Using ISE iMPACT

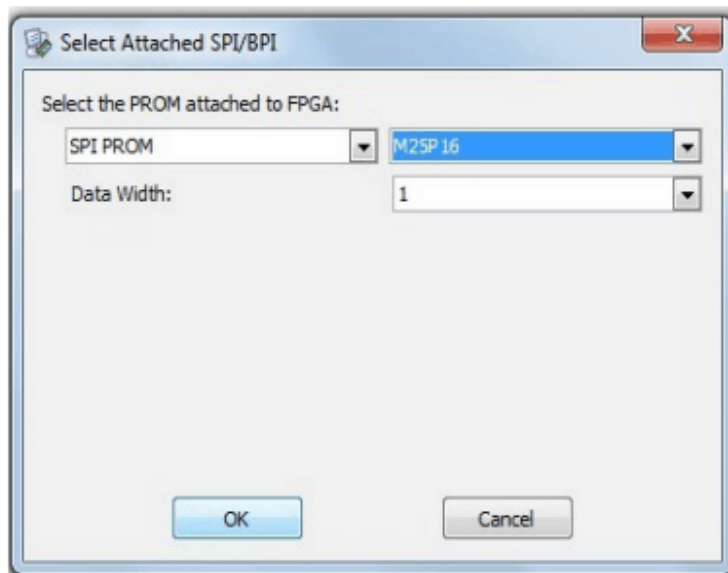
Step 1: Make sure Xilinx Platform Cable USB is connected properly to the board. Open ISE iMPACT. Click on “Boundary Scan” in the iMPACT flows window in the left top corner. Then right click on the window panel in the right side. Select “Initialize Chain”.



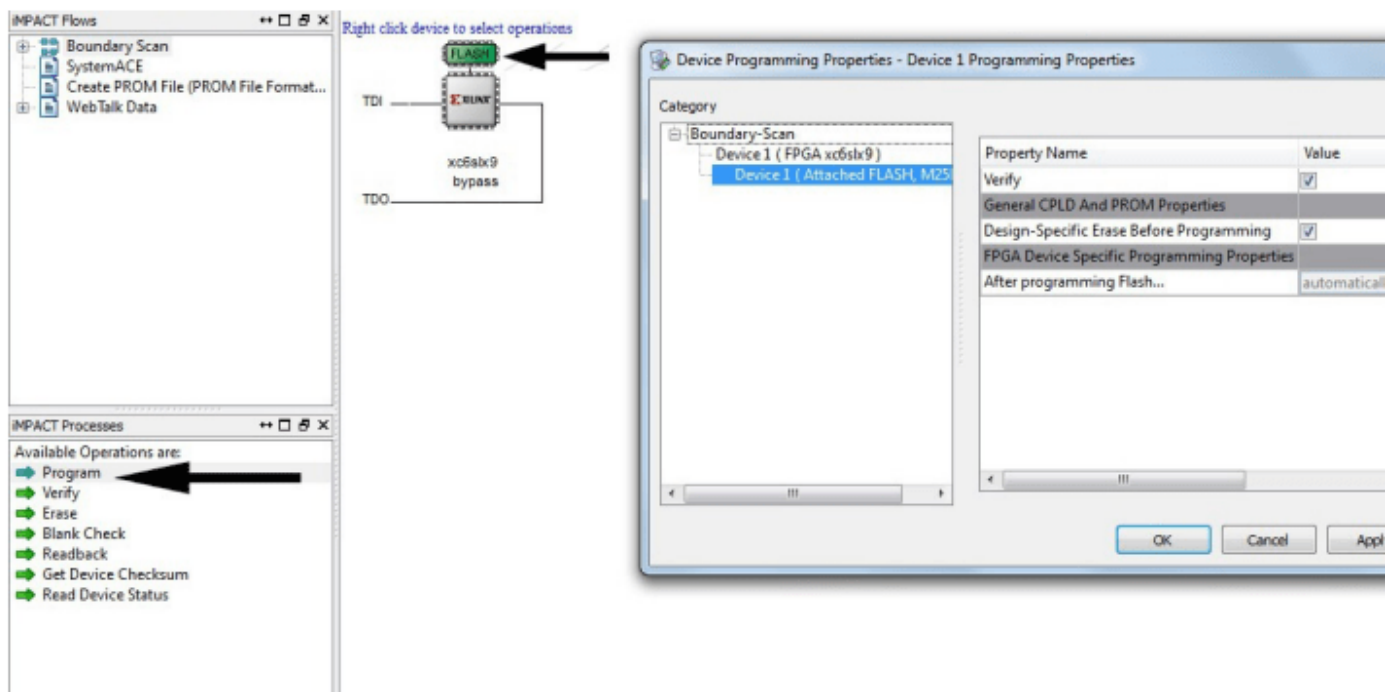
Step 2: If the device is detected properly you will get a pop up window as shown below, Click OK. Then right click on the SPI/BPI (next to the black arrow in the below fig.), select Add SPI/BPI Flash.



Step 3: Select the “.mcs” file we already created and click OK. Now choose “M25P16” in the dialogue box appeared, then click OK.



Step 4: Click on “Flash”, Double Click on Program, select OK. If the programming is successful, a confirmation message will be displayed.



7 CODING
CODE FOR NORAML CLOCK BCD ADDER

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity Bcd_adder is
    port( Clock_enable_B: in std_logic;
          Clock: in std_logic;
          Reset: in std_logic;
          Output: out std_logic_vector(0 to 3));
end Bcd_adder;

architecture Behavioral of Bcd_adder is
    signal temp: std_logic_vector(0 to 3);
begin
    process(Clock,Reset)
    begin
        if Reset='1' then
            temp <= "0000";
        elsif(rising_edge(Clock)) then
            if Clock_enable_B='0' then
                if temp="1001" then
                    temp<="0000";
                else
                    temp <= temp + 1;
                end if;
            end if;
        end if;
    end process;
    Output <= temp;
end Behavioral;

```

CODE FOR GATED CLOCK BCD ADDER

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity bcd_adder is
    port(
        a,b : in unsigned(3 downto 0); -- input numbers.
        carry_in : in std_logic;

```

```

        sum : out unsigned(3 downto 0);
        carry : out std_logic
    );
end bcd_adder;

```

architecture arch of bcd_adder is

begin

```

process(a,b)
variable sum_temp : unsigned(4 downto 0);
begin
sum_temp := ('0' & a) + ('0' & b) + ("0000" & carry_in);
if(sum_temp > 9) then
    carry <= '1';
    sum <= resize((sum_temp + "00110"),4);
else
    carry <= '0';
    sum <= sum_temp(3 downto 0);
end if;
end process;

```

end arch;

8. OUTPUTS

8.1. GATED CLOCK

2.3. Power Supply Summary

Power Supply Summary				
	Total	Dynamic	Static	Power
Supply Power (mW)	13.65	0.00	13.65	

8.2 NORMAL CLOCK

2.3. Power Supply Summary

Power Supply Summary				
	Total	Dynamic	Static	Power
Supply Power (mW)	14.11	0.45	13.66	

9. CONCLUSIONS AND FUTUREWORK

In this paper, we considered clock gating for power reduction in FPGAs and propose that clock gating capability be incorporated at various points in the clock distribution network. We presented a placement algorithm that incorporates clock power and locates clock loads accordingly to take advantage of clock gating architectural enhancements. To our knowledge, this work represents the first study of clock gating architectures in FPGAs. Results show that depending on the architecture and clock activity, clock power can be reduced by over 50%.

Future work will involve incorporating the clock gatingaware placement algorithm into a complete power-awareFPGA CAD flow, allowing trade-offs between other power-aware tool phases to be explored. Another future direction involves modeling clock gating within the VPR router to permit extraction of accurate post-routing critical path delay and interconnect capacitance. It will also be important to understand the impact of gating on timing analysis to ensure functional correctness is maintained when enable signals are migrated from flip-flops to enable pins on clock network switches.